artifact-to-github-pages: how the deploy works

Nov 9, 2026 · @p

What it does

The skill takes a Claude artifact (an HTML page or a React app) and puts it live as a GitHub Pages site on your own account, then hands back the `github.io` link. Claude does every step itself through the Claude in Chrome tab where you are logged into GitHub. You never upload, drag or download a file.

Start it with `/artifact-to-github-pages` plus what you want, for example:

- "host this on my github" for a first deploy
- "publish the changes to github" to update a site that is already live

The only question it may ask is a one-time privacy check: a Pages site is public, so if the page holds internal metrics, names or phone numbers it asks once whether to publish as is.

Ground rules

These hold on every run and override anything else the skill could try.

- **Never ask you to upload anything.** Chrome's `file_upload` won't take files from Claude, so the skill doesn't try it.
- **No relay hosts.** Files never pass through Google Drive, Gist, Pastebin or any other host, even if a connector for one is available.
- **No tokens or passwords.** Your logged-in Chrome does the authenticated work. If GitHub asks you to sign in again, that step is yours.
- **Buttons by `find`, never by screen coordinates.**
- **Only the deploy's own repo.** It never deletes repos or touches other ones.
- **Two Chrome timeouts in a row = stop.** It reports which step finished and what's left, and waits for you to say "go".

First deploy, start to finish

1. **Find the artifact and name the repo.** It reads the published artifact or the file from the chat, picks a short hyphenated repo name and a one-line description. It asks the privacy question only if the page holds internal or personal data.

2. **Build the static site.** `build_site.py` turns the artifact into `index.html` (plus `bundle.js` for React). A page with inlined images and video stays one self-contained file.
3. **Create the repo.** At github.com/new it fills the name and description, keeps it Public (free Pages needs that), and reads typed fields back to catch dropped characters.
4. **Turn Pages on.** Settings → Pages → Source → **GitHub Actions**, done before the first workflow commit so that run deploys.
5. **Put the files in.** Small files go straight in with Route 1; anything bigger goes in chunks plus an assemble workflow (Route 2). See the next section.
6. **Verify.** Each commit is read back and hash-checked, the Actions run must be green, and the live URL is checked in the browser.
7. **Tidy.** It ticks "Use your GitHub Pages website" in the repo's About box and closes the tabs it opened.

The final message is the live URL first, one line on what was done, and any caveats that apply (for example, browser storage is per device off claude.ai).

The five routes into GitHub

Each file takes the cheapest route that fits it. All five run through GitHub's own pages in your logged-in Chrome.

Route	Use it for	How it works
1 · Prefilled editor	Small text files: README, config, a workflow, HTML under ~4 KB	Opens `github.com/<user>/<repo>/new/main?filename=<path>&value=<url-encoded content>` (whole URL under ~6,000 characters). GitHub opens the editor already filled in; Claude clicks **Commit changes…**, then **Commit changes** in the dialog. A new-file URL can't overwrite an existing path.
2 · Chunks + assemble	Bigger files	gzip → base64 → ~4,400-character chunks committed with Route 1 as `.github/upload/p01.txt`, `p02.txt`, … Then `.github/workflows/assemble.yml` joins them in order, decodes, checks the sha256 against a hardcoded hash, writes the real file, removes the chunks and commits as github-actions[bot].

Route	Use it for	How it works
3 · Fetch in the workflow	Big images or video already at a public URL (e.g. raw.githubusercontent.com of another public repo)	Not sent at all: the workflow `curl`s them, and can run ffmpeg or Pillow on them.
4 · Patch	Updating a file that's already in the repo	Only the changes are sent, as [prefix anchor, suffix anchor, new text] in a gzipped base64 env var. The workflow checks the old file's sha256, applies the changes and checks the result's sha256 before committing. If the workflow URL would be too long, the patch goes in as a Route 2 chunk instead.
5 · Pages deploy	Every workflow, once Source is set to GitHub Actions	The workflow ends with `configure-pages`, `upload-pages-artifact` (path `_site`) and `deploy-pages`.

Each chunk costs three browser calls (open the URL, click Commit changes…, click Commit changes), so the skill keeps payloads small: a 4 MB page went up as 12 chunks; later updates as 1.

Updating a site that's already live

An update never resends the page. It goes like this:

1. **Get the repo's current file** from raw.githubusercontent.com, not from memory.
2. **Build the target from the repo's own media.** Inline images and videos that haven't changed are reused byte for byte from the repo's file, so the diff only holds text.
3. **Make the patch.** `make_patch.py` diffs old and new by tokens (tags, text runs, inline media), so a one-line HTML page with big inline images still gives a small patch. It dry-runs the exact apply code locally and refuses unless the result matches the new file byte for byte.
4. **Generate the workflow** with a fresh name each time (`patch-1`, `patch-2`, …), since the editor can't overwrite an existing workflow file.
5. **Commit the chunk (if any), then the workflow.** Pushing the workflow starts the run.

New media that isn't public anywhere (a re-rendered video, a newly cropped screen) is rebuilt inside the workflow from material already in the repo: media embedded in the current page (matched by hash), fonts from npm. Only the page text, a small build script

and tiny inputs are sent. Re-encoded media can't match byte for byte, so the check is the page text's hash with media stripped out.

How each step is checked

When	Check	Tool
After every commit	The file read back through raw.githubusercontent.com (with a cache-buster) matches the local copy by sha256	`verify_raw.py`
Before the workflow runs	The joined chunks match the payload hash hardcoded in the workflow	the workflow itself
During the run	The repo's file is the expected old version; the result matches the expected new hash (or text hash, when media is rebuilt)	the workflow itself
After the run	The Actions tab shows the run green, including the deploy step	Chrome
Live page	Sections, `video.readyState` and image loads checked in the page; the live file hashed in the browser and compared with the expected hash	Chrome

The browser hash matters because raw.githubusercontent.com can keep serving the previous `main` for a few minutes after a workflow commit. The live page is the source of truth.

When things stall

Situation	What the skill does	What you do
Chrome bridge times out twice in a row	Stops, lists which step finished and what's left. Anything checkable from its sandbox (raw hashes) it checks while waiting	Check the Claude Desktop app for a pending prompt or restart it, then say "go"
Chrome tools not available this turn	Prepares and dry-runs the patch and workflow, then waits	Reconnect the Claude in Chrome extension and say "go"

Situation	What the skill does	What you do
Turn runs out of tool calls mid-upload	Stops cleanly after the last verified chunk	Say "go"; it resumes from the next chunk
A commit doesn't verify	Re-commits that one file; never moves on with a mismatch	Nothing
Live page still looks old	Hashes the live file in the browser; raw's cache can lag a few minutes	Hard-refresh the tab

It never falls back to asking you to drag files or to a relay host.

Scripts in the skill

Script	What it does	Key options
`build_site.py`	Turns the artifact into a static site (`index.html`, plus `bundle.js` for React) and lists caveats	`--title`, `--desc`
`make_chunks.py`	gzip + base64 + ~4,400-character chunks; writes one ready Route 1 URL per chunk and the file's sha256	`--repo user/repo`, `--chunk`
`make_patch.py`	Token diff of old vs new into anchor triples; writes the env-var payload and the apply code; dry-runs it locally	exits non-zero unless the dry run is byte-identical
`make_workflow.py`	Writes the assemble or patch workflow and its commit URL; refuses if the URL would pass ~6,000 characters	`assemble` / `patch`, `--pages`, `--name`, `--patch-chunks`, `--fetch "url path"`, `--run "cmd"`
`verify_raw.py`	Reads committed files back through raw.githubusercontent.com and compares sha256	`--strip` for chunk files

Inside a generated workflow, the old target is kept at `/tmp/old.html` and every file sent in `.github/upload/` is kept in `/tmp/upload/`, so `--run` steps (a regen script, ffmpeg, Pillow) can use them after the chunks are cleaned up.